Vicente Rendo

FSFEUP Electronics & Software (Embedded Software)
Decision Phase Report

October 29, 2026

Contents

1 Introduction **3**
- 1.1 The **Context** class . . . 3
- 1.2 Program flow . . . 3

2 State Machines **4**
- 2.1 Namespacing and Manager classes . . . 4
- 2.2 The Session System . . . 4
  - 2.2.1 Activation . . . 4
  - 2.2.2 Active Monitoring . . . 5

3 The Signal Processing Pipeline **6**
- 3.1 Signal Processors . . . 6
- 3.2 Signal Processor Chains . . . 6

4 Fault Management **7**
- 4.1 Detection . . . 7
  - 4.1.1 Fault Detection Signal Processors . . . 7
  - 4.1.2 Context Methods . . . 7
- 4.2 Transmission . . . 7

5 User Interface **8**
- 5.1 Fundamentals . . . 8
- 5.2 Overview . . . 8
- 5.3 The Faults Panel . . . 9

6 Noteworthy Remarks **10**
- 6.1 Anti-BSPD Trigger . . . 10
- 6.2 Braking Deviation Detection . . . 10
- 6.3 Speed/Acceleration Estimation from Wheel Rotation Speed . . . 10
  - 6.3.1 Wheel Rotation Speed Measurement . . . 10
  - 6.3.2 Estimating Speed and Acceleration . . . 10
- 6.4 Unstable Signals and Loose Wiring . . . 11

A R2D/ATS Specifics **12**
- A.1 Ready-to-Drive (R2D) . . . 12
  - A.1.1 Session Stages . . . 12
  - A.1.2 R2D Button Events . . . 12
- A.2 Activate Tractive System (ATS) . . . 13
  - A.2.1 Session Stages . . . 13
  - A.2.2 ATS Button Events . . . 13

B Class Diagram **14**

1 Introduction

Let's start with a quick overview of the code, to make it easier to follow along:

1.1 The Context class

The `Context` class (and it's single instance) represent the program execution context. `Context` holds the program's state, as well as some methods with which to interact with that state. Having a centralized object like this allows for better flexibility, as the entire program's state can be referenced into any scope, allowing, for example, state machines to freely and directly inspect the entire execution context through a single reference, rather than relying on fine-tuned individual references.

1.2 Program flow

Even though most code lives in the `Context` class, the overall flow of the program is controlled through the main `loop()` function. Methods called from `loop()` tend to represent larger objectives (e.g. `read_sensors()` or `update_faults()`) rather than individual actions.

Inside `Context`, however, things get more specific. The idea here is to name and isolate every little action that needs to be taken — every sensor read, actuator setting and so on deserves it's own small and simple method. This makes it easier to analyse exactly when each thing happens and keeps behavior consistent across all parts of the program, as well as having the code explain itself with method names rather than having to comment everywhere.

These principles make future troubleshooting a whole lot faster, as fixing faulty behavior becomes as easy as modifying a few intuitively-named functions, instead of having to find every time that behavior is summoned throughout the enitre code base.

2 State Machines

The two standalone state machines in this program are those governing the **Ready to Drive (R2D)** and **Activate Tractive System (ATS)** modes. For the most part, these two behave in extremely similar ways, so anything up next can be applied to both, unless noted otherwise. Refer to Appendix A for details on the individual implementation of the following principles on each state machine.

2.1 Namespacing and Manager classes

Each state machine has it's own namespace and "manager" class (`ats::ATS` and `r2d::R2D`). These are responsible for holding the logic that governs their respective state, described throughout the rest of this section (hereby referred to as *state logic*), and work as independently as possible from the rest of the program, interfacing with the execution context (e.g. reading from hardware or checking other states) only when necessary, through references to the `Context` instance.

To execute state logic, each manager class defines a `tick()` method. It is inside this function that stages are executed and checks are performed. You may think of it as the state machine's own `loop()` function.

2.2 The Session System

A new session is started on a mode every time its activation is requested. For example, every time Ready-to-Drive activation is called, a new Ready-to-Drive session is generated. Sessions provide a way of tracking the behavior of a state machine with the use of **stages**, grouped together into different **phases** — "Idle", "Activating" and "Active".

2.2.1 Activation

The "Activating" phase kicks in once activation of a mode is called. This **phase** consists of multiple **stages**. Which stages make up the activation phase depends on the individual state machine. These stages work to set and confirm conditions required for the activation of its respective mode. The following diagram outlines the different stages in the activation phase of the ATS mode:

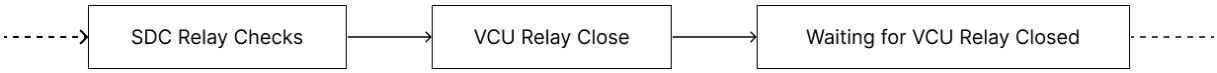


Figure 1: ATS mode Activation phase

Each of these stages plays its own role and may either advance the activation process to the next stage once complete, or halt activation with a failure.

Activation Timeout Both R2D and ATS define a period of time since the beginning of an activation after which the activation times out. This is useful when dealing with stages that wait for a condition to be met that is outside the program's control.

Parallel Checks [R2D only] Some conditions need to remain met throughout the whole of the activation phase and are checked **continuously** from beginning to end, in **parallel** with other stages, regardless of which stage is currently active.

2.2.2 Active Monitoring

Checks are continuously performed as long as a state is in the *Active* phase, to ensure it may remain active. These usually include the same checks from the Parallel Checks section. If active monitoring detects that a required condition is no longer met, the state is deactivated with the corresponding error.

3 The Signal Processing Pipeline

Most raw signals from the many sensors connected to the MCU need to be processed and transformed in some way. This process, and its many components, comprise the Signal Processing Pipeline. Code definitions regarding the Signal Processing Pipeline are located in the `/src/signal` subdirectory.

3.1 Signal Processors

Signal Processors (**SP**s from now on) receive a value, do something with it, and make the result available for reading. The `SignalProcessor` class declares an interface for all signal processors to implement.

On a technical note, you'll notice I said "available for reading", rather than something like "is returned". This is because an SP can be updated with `update()`, but it's result is only retrievable through its `out()` method. This makes it so the result of a single update can be read multiple times from multiple parts of the code. SPs also have an `in()` method, for retrieving the latest input value with influence on the output.

It's also important to point out that SPs do more than just unit conversion and standard filters like Moving Average. A big part of all Fault Detection is also handled through these, but we will cover that later.

3.2 Signal Processor Chains

Signal Processor Chains (**SPC**s) are a way of (you guessed it) chaining Signal Processors together, so that each SP is updated with the output of the one before. SPCs are themselves Signal Processors and implement the `SignalProcessor` class. The output of an SPC is the output of the last SP in the chain.

Most (if not all) sensors get their signals processed through SPCs. A typical SPC is made up of an `AdcVolts` SP, which converts an ADC's 16-bit unsigned integer to a 0V to 3.3V value, chained with an `AdcRangeVerifier`, which checks that voltage and emits a fault when appropriate.

4 Fault Management

Fault Management can be broken down into two parts: **Detection** and **Transmission**.

4.1 Detection

Detection is achieved in two different ways, depending on the fault.

The FaultHelper class `FaultHelper` tracks a fault condition over time and emmits a fault only when something changes. It also works on keeping those faults sufficiently spaced-out as to not overload the faults queue (and the engineer at the other end) with the same issue over and over again. It is used by both of the following detection strategies.

4.1.1 Fault Detection Signal Processors

As mentioned earlier, Signal Processors are used for more than just filtering and converting units. Turns out they also make the perfect platform for reactivity - after all, it's just a piece of code that runs when a value is updated. Fault detection SPs check for a fault condition from within the SP Chain. SP fault detection is used for more "generic" faults that can be measured in a similar way amongst many different sensors, such as `UNSTABLE_SIGNAL` or `OUT_OF_RANGE`.

4.1.2 Context Methods

The other way faults are detected is through methods in the `Context` class, called from `update_faults()`. This approach is used for more specific faults that only apply to a unique part of the system, like `BRAKE_DEVIATION` or `SDC_IMPLAUSIBILITY`.

4.2 Transmission

Once detected, a `Fault` object is added to the `Context`'s `faults` queue. This queue is later read from in the `update_can()` method, which takes faults one by one and transmits them to `/CAN/MCU_OUT` in these 5 fields:

fault_n	Pseudo-random integer. Changes on every new fault to signal an update.
fault_severity	Integer representing the fault's severity.
fault_type	Integer representing the fault's type.
fault_arg	Unsigned 32-bit integer containing bit-encoded fields specific to that fault type.
fault_detected_at	Unsigned 64-bit integer representing milliseconds since epoch at fault detection.

These fields are initialized with 0 at start-up to prevent the schema from changing mid-run and causing Foxglove to freak out.

5 User Interface

5.1 Fundamentals

Designing a UI is optimizing between ease-of-use/readability and efficiency/amount of information. Setting a target balance between these two is key for a successful interface. Opposite to UI's made for mass use, like social media apps, our context here is better comparable to a cashier's desk, where the objective is less that someone can pick it up for the first time and intuitively understand it, but instead that everything is as fast and predictible as possible for the **trained** user. Having lots of plots and widgets in the same page is usually encouraged when designing for efficiency, as opposed to when optimizing for ease-of-use.

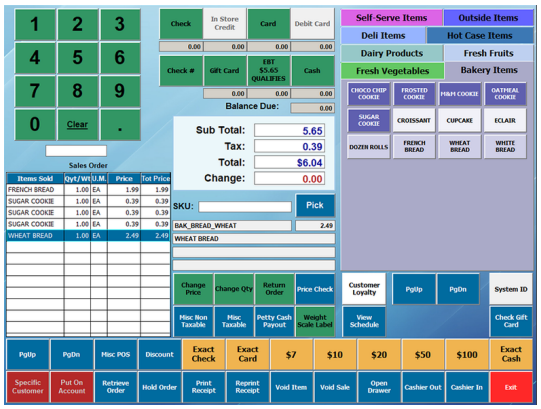


Figure 2: Cashier Point-of-Sale System

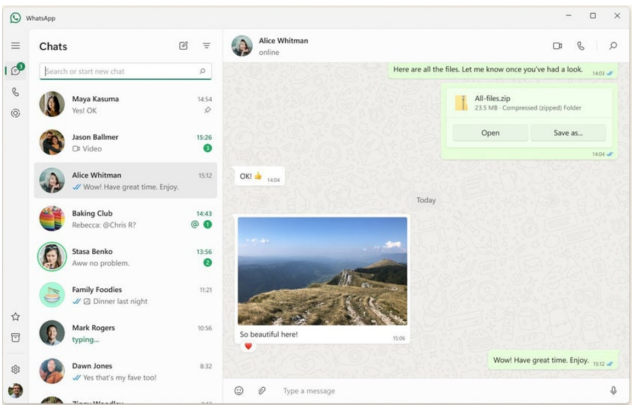


Figure 3: WhatsApp Web

However, that does not make it okay to scatter everything across the screen without any thought. The need for *some* intuitiveness and ease-of-use is still there, it is just not as prevalent.

5.2 Overview

Here is how it turned out. The layout is split into 3 columns:

1. Faults Uses a custom extension with a React component to display all received faults.

2. Visualization Contains most needed plots, divided into two sub-columns. The first sub-column plots braking and throttle/torque, while the second sub-column plots wheel measurements and acceleration. Note that each sub-column is contained inside it's own scrollable *Stack* panel, adding the possibility for more quickly-accessible plots in the future. On the bottom, collapsed in Figure 4, you'll also find the SDC Drawer, with useful history of SDC sensor readings. The chart on the left side of the drawer plots extra details on the extra-important SDC VCU Sensor, used by the VCU to judge whether the SDC is closed or not.

3. State The third column holds mode states as well as full messages from the `/CAN/MCU_OUT` topic. The row with 4 indicators represent different actuators and take their state directly from VM topics, so, in practice, these would probably not exist.

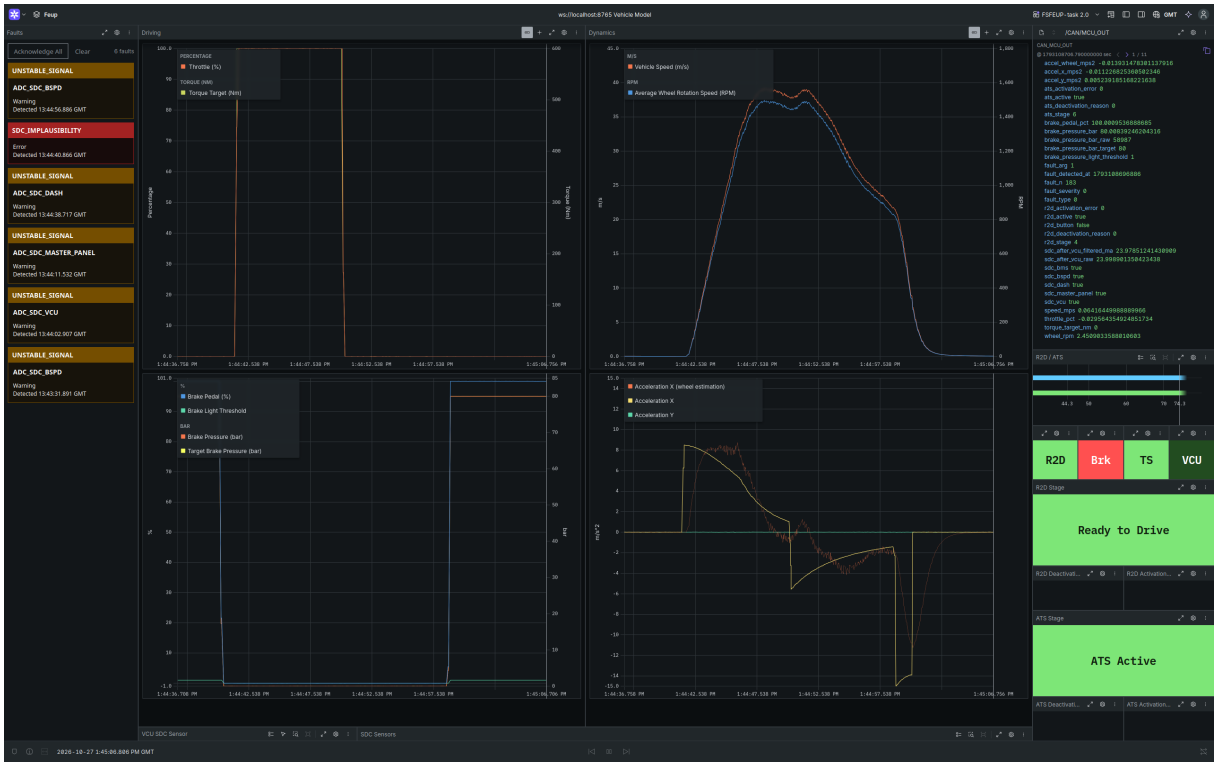


Figure 4: Finalized Foxglove Panel Interface

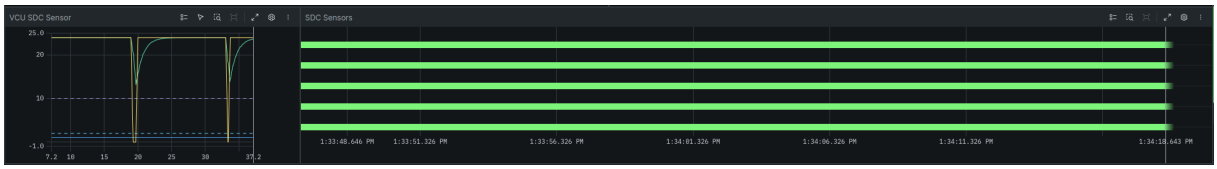


Figure 5: SDC Drawer

5.3 The Faults Panel

I made the faults panel extension specifically for this project. It scans `/CAN/MCU_OUT` for faults, logs them, and displays that list as seen on the left side of Figure 4. *Warning* faults are displayed in yellow, while *Error* faults are shown in red. Additionally, you can click on a fault to acknowledge it, making it less attention-grabbing.

6 Noteworthy Remarks

6.1 Anti-BSPD Trigger

Instead of just limiting torque as long as brake pressure is above a certain threshold and calling it a day, I had the idea of having it cut off torque only just before the 500ms implausibility limit, waiting some 50ms or so and then releasing it again, repeating these short cutoffs for as long as necessary, kind of like an ABS. However, although this works with an "ideal" BSPD, this may not work in practice, given the BSPD must be a non-programmable circuit and may be using a capacitor for timing, which may not have time to (dis)charge in-between pulses.

6.2 Braking Deviation Detection

Part of the mandatory faults to implement was a "brake implausibility condition", defined as:

(...) a deviation exceeding $\pm 5\%$ *from normal operating correlation (...)*

I interpreted this condition as follows:

$$\frac{|P_{\text{measured}} - P_{\text{expected}}|}{P_{\text{measured}}} > 0.05$$

However, anything below 2-5 bar of pressure could start generating faults purely due to noise in sensor readings. My solution was to inhibit this fault below 5 bar of measured pressure.

6.3 Speed/Acceleration Estimation from Wheel Rotation Speed

6.3.1 Wheel Rotation Speed Measurement

Wheel RPM is measured through the `Rpm` Signal Processor. Initially, I had the SP update it's output to $\frac{1}{\text{minutes since last update}}$ every time it detects the start of a pulse. However, because the sensor does not produce pulses below a certain speed, it could get stuck reporting speeds that were higher than actually possible. To fix this, the proceessor now uses the **largest** of the following time values in the equation:

1. time between the latest update and the one before it;
2. time since the latest update.

This works because, for example, if 2 minutes have passed since the last pulse, then the interval until the next pulse must be at least 2 minutes. So the next reading, $\frac{1}{\text{minutes since last update}}$, will be guaranteed to be at most $\frac{1}{2}$ revolutions per minute.

6.3.2 Estimating Speed and Acceleration

With rotation speed sorted, computing speed is a matter of multiplying RPM by the circumference of the wheels and dividing by 60 to get m/s. Acceleration is simple "real-time differentiation" — Measure how much the speed has changed and over how much time. However, although this does yield get a sensible result, it's so smoothed from all the filtering that it's mostly useless. One of the graphs in Figure 4 plots this estimation.

6.4 Unstable Signals and Loose Wiring

I realised pretty soon into development that something was up with the SDC line sensors, as they kept individually cutting off for up to a second at random times and intervals. This is usually a sign of a loose connection, but short drop-off's like these shouldn't immediately deem the entire SDC closed and possibly ruin a run, so I added `MovingAverage` and `Threshold` to the SPCs, as well as a fault detector (`SignalStabilityVerifier`) that spawns a fault every time a sensor goes from reading above 20V and back in less than 1000ms.

A R2D/ATS Specifics

A.1 Ready-to-Drive (R2D)

Activation Timeout	10 seconds
Parallel Checks	ATS mode active
Active Requirements	ATS mode active

A.1.1 Session Stages

Phase	Stage	Description
Idle	Inactive	R2D not active. Default stage for new sessions.
Idle	Deactivated	R2D not active. Previous activation was successful.
Idle	Activation Failed	R2D not active. Previous activation was **not** successful.
Activation	Waiting for Pedals	Waiting for throttle idle and brakes pressed.
Active	Active	R2D active.

A.1.2 R2D Button Events

Event	Condition	Effect
Hold	Phase	Begin activation.
Hold	Active	Deactivate. (`DeactivationReason::User`)
Click	Activating	Cancel activation. (`ActivationError::User`)
Click	Idle	Reset session.

A.2 Activate Tractive System (ATS)

Activation Timeout	10 seconds
Parallel Checks	SDC Closed
Active Requirements	SDC Closed

A.2.1 Session Stages

Phase	Stage	Description
Idle	Inactive	ATS not active. Default stage for new sessions.
Idle	Deactivated	ATS not active. Previous activation was successful.
Idle	Activation Failed	ATS not active. Previous activation was **not** successful.
Activation	SDC Relay Checks	Checking all relays in the SDC are closed, except for VCU.
Activation	VCU Relay Close	Actuating (closing) VCU SDC relay.
Activation	Waiting for SDC Closed	Waiting for SDC to be fully closed.
Active	Active	ATS active.

A.2.2 ATS Button Events

Event	Condition	Effect
Hold	Phase	Begin activation.
Hold	Active	Deactivate. (`DeactivationReason::User`)
Click	Activating	Cancel activation. (`ActivationError::User`)
Click	Idle	Reset session.

B Class Diagram

`Context()`
Holds program-wide execution context

`Button()`
Helps with physical button event handling

`ats` namespace

```
ATS()

Session {}

SessionStage : uin8_t
ActivationError : uin8_t
DeactivationReason : uin8_t
```

`r2d` namespace

```
R2D()

Session {}

SessionStage : uin8_t
ActivationError : uin8_t
DeactivationReason : uin8_t
```

`SignalProcessor()`

```
SignalProcessorChain()

AdcVolts()
MovingAverage()
SignalStabilityVerifier()
...
```

`fault` namespace

```
FaultHelper()

Fault {}
FaultBuilder {}

Type : uin8_t
Severity : uin8_t
```